



Université de Bourgogne
UFR Sciences et Techniques

Modélisation et Animation de Pikachu

Projet de Synthèse d'Image

Licence 3 Informatique

Binôme : LAWSON Ryan et MORAUX Thomas

Encadrant : S. Languetin



Table des matières

1	Introduction	3
1.1	Contexte du projet	3
1.2	Objectifs	3
2	Modélisation de Pikachu	3
2.1	Structure générale	3
2.2	Primitive paramétrique	4
2.3	Gestion des couleurs et matériaux	4
3	Gestion des textures	4
3.1	Chargement des images JPEG	4
3.2	Texture plaquée sur les sphères	5
3.3	Texture enroulée autour du cylindre	5
3.4	Gestion des textures manquantes	5
4	Éclairage	5
4.1	Types de lumières implémentées	5
4.1.1	Lumière principale (LIGHT0)	6
4.1.2	Lumière secondaire (LIGHT1)	6
4.2	Interaction lumière/matériaux	6
5	Animations	6
5.1	Animation automatique	6
5.1.1	Rotation de la scène	6
5.1.2	Balancement de Pikachu	6
5.2	Contrôles manuels	7
6	Navigation et interaction	7
6.1	Système de caméra orbite	7
6.1.1	Zoom (touches Z/z)	7
6.1.2	Rotation (flèches et souris)	7
6.2	Gestion des événements	7
7	Architecture du code	8
7.1	Organisation des fichiers	8
7.2	Structure de la classe Pikachu	8
7.3	Fonctions principales	8
8	Résultats et difficultés	9
8.1	Résultats visuels obtenus	9
8.2	Difficultés rencontrées	9
8.2.1	Coordonnées de texture sur les sphères	9
8.2.2	Gestion hiérarchique des transformations	9

8.2.3	Synchronisation des animations	9
8.3	Solutions apportées	9
9	Conclusion	10
9.1	Bilan du projet	10
9.2	Améliorations possibles	10
9.3	Retour d'expérience	10
10	Annexes	10
10.1	Extraits de code significatifs	10
10.1.1	Génération de sphère paramétrique	10
10.1.2	Gestion des textures JPEG	11
10.2	Références bibliographiques	11

Chapitre 1

Introduction

1.1 Contexte du projet

Ce projet de synthèse d'image, réalisé dans le cadre de la Licence 3 Image 2025, a pour objectif la modélisation, l'habillage et l'animation du personnage Pikachu. Le projet met en œuvre les concepts fondamentaux de la synthèse d'image : modélisation géométrique, application de textures, éclairage et animation.

1.2 Objectifs

Les objectifs principaux de ce projet sont :

- Modéliser Pikachu en utilisant des primitives géométriques complexes
- Implémenter au moins une primitive paramétrique (sphère)
- Appliquer deux textures différentes (une plaquée et une enroulée)
- Gérer deux types de lumières avec des propriétés distinctes
- Implémenter un système de navigation caméra complet
- Créer des animations fluides (automatique et manuelle)
- Structurer le code de manière modulaire et maintenable

Chapitre 2

Modélisation de Pikachu

2.1 Structure générale

La modélisation de Pikachu repose sur une composition hiérarchique de primitives géométriques pour créer les différentes parties du corps :

- **Corps** : Sphère principale légèrement écrasée selon l'axe Z (facteur 0.8)
- **Tête** : Sphère positionnée au-dessus du corps avec un facteur d'échelle de 0.85
- **Oreilles** : Cylindres allongés et inclinés avec parties intérieures noires
- **Pattes** : Sphères de différentes tailles selon leur position (avant/arrière)
- **Queue** : Structure composée de segments quadrilatères avec bout triangulaire noir
- **Visage** : Yeux, joues, nez et bouche modélisés séparément avec des sphères et triangles

2.2 Primitive paramétrique

La primitive paramétrique principale utilisée est la **sphère**, générée à l'aide des équations paramétriques suivantes :

$$x = r \cdot \cos(\theta) \cdot \sin(\phi)$$

$$y = r \cdot \sin(\theta) \cdot \sin(\phi)$$

$$z = r \cdot \cos(\phi)$$

Où :

- r est le rayon de la sphère
- $\theta \in [0, 2\pi]$ est l'angle azimutal (longitude)
- $\phi \in [0, \pi]$ est l'angle polaire (latitude)

Cette implémentation permet de contrôler le niveau de détail via le paramètre **segments**. Pour une sphère de 24 segments, nous obtenons un bon compromis entre qualité et performance.

2.3 Gestion des couleurs et matériaux

Les couleurs sont gérées de manière différenciée selon les parties :

- **Corps et tête** : Jaune caractéristique (1.0, 0.85, 0.0) avec texture de peau
- **Oreilles** : Noir pour l'intérieur (0.1, 0.1, 0.1), jaune pour l'extérieur
- **Joues** : Rouge rosé (0.9, 0.3, 0.2) pour les cercles caractéristiques
- **Yeux** : Noir avec reflets blancs pour donner de la vie au regard
- **Queue** : Jaune avec bout noir en forme d'éclair
- **Ceinture** : Texture rayurée ou couleur rouge fallback

Les matériaux utilisent les propriétés spéculaires standard d'OpenGL avec un coefficient de brillance de 50, ce qui donne un aspect légèrement brillant à la peau de Pikachu.

Chapitre 3

Gestion des textures

3.1 Chargement des images JPEG

Le chargement des textures se fait via la bibliothèque **libjpeg**. La fonction **chargerImageJpeg** gère l'ensemble du processus :

- Ouverture et vérification du fichier
- Décompression JPEG avec gestion d'erreurs
- Allocation mémoire pour les données pixels
- Création de la texture OpenGL avec paramètres appropriés

Les paramètres de texture sont configurés pour le filtrage linéaire et le wrapping repeat, permettant une application souple sur les différentes primitives.

3.2 Texture plaquée sur les sphères

La texture principale (`peau.jpg`) est appliquée sur les faces du corps de Pikachu à l'aide des coordonnées de texture générées lors du maillage des sphères. Le mapping UV est calculé automatiquement lors de la génération des sommets :

```
1 float u = (float)j / segments;
2 float v0 = (float)i / segments;
3 glTexCoord2f(u, v0);
4 glVertex3f(radius * x * zr0, radius * y * zr0, radius * z0);
```

Cette approche assure une répartition uniforme de la texture sur toute la surface de la sphère.

3.3 Texture enroulée autour du cylindre

La deuxième texture (`rayures.jpg`) est appliquée en mode enroulé autour du cylindre de la ceinture. L'implémentation utilise un quad strip avec coordonnées de texture progressives :

```
1 for (int i = 0; i <= segments; i++) {
2     float angle = i * angleStep;
3     float x = cosf(angle);
4     float z = sinf(angle);
5     float u = (float)i / segments;
6     glTexCoord2f(u, 0.0f); glVertex3f(x * radius, -height/2, z * radius);
7     glTexCoord2f(u, 1.0f); glVertex3f(x * radius, height/2, z * radius);
8 }
```

Cette technique crée un effet de texture continue autour du cylindre, idéal pour les motifs répétitifs comme les rayures.

3.4 Gestion des textures manquantes

En cas d'échec du chargement des textures, le système génère automatiquement une texture de secours :

- Texture damier noir et blanc pour la peau
- Couleur unie rouge pour la ceinture
- Message d'avertissement dans la console

Cette approche robuste permet au programme de fonctionner même avec des ressources manquantes.

Chapitre 4

Éclairage

4.1 Types de lumières implémentées

Le système d'éclairage utilise deux sources de lumière distinctes pour créer une ambiance réaliste :

4.1.1 Lumière principale (LIGHT0)

- Position : (3.0, 5.0, 3.0) - lumière directionnelle haute
- Diffuse : Blanc (0.9, 0.9, 0.9) - éclairage principal
- Ambiance : Gris moyen (0.3, 0.3, 0.3) - éclairage d'ambiance
- Spéculaire : Blanc pur (1.0, 1.0, 1.0) - reflets brillants

4.1.2 Lumière secondaire (LIGHT1)

- Position : (-3.0, 3.0, -3.0) - contre-jour
- Diffuse : Bleuâtre (0.5, 0.5, 0.7) - ambiance froide
- Ambiance : Bleu très sombre (0.1, 0.1, 0.2) - ombres colorées

Cette combinaison crée un éclairage tridimensionnel qui met en valeur les formes de Pikachu.

4.2 Interaction lumière/matériaux

Les matériaux sont configurés pour interagir naturellement avec les sources lumineuses :

```
1 GLfloat matSpecular[] = { 0.5f, 0.5f, 0.5f, 1.0f };
2 GLfloat matShininess[] = { 50.0f };
3 glMaterialfv(GL_FRONT, GL_SPECULAR, matSpecular);
4 glMaterialfv(GL_FRONT, GL_SHININESS, matShininess);
```

L'activation de `GL_COLOR_MATERIAL` permet de combiner les couleurs des sommets avec l'éclairage, essentiel pour les parties non texturées.

Chapitre 5

Animations

5.1 Animation automatique

L'animation automatique comprend deux composantes :

5.1.1 Rotation de la scène

La scène entière tourne automatiquement autour de l'axe Y à vitesse constante (0.5 degré par frame), permettant d'observer Pikachu sous tous les angles.

5.1.2 Balancement de Pikachu

Pikachu effectue un léger mouvement de balancement basé sur une fonction sinus :

```
1 animationAngle += 2.0f;
2 glRotatef(sin(animationAngle * M_PI / 180.0f) * 5.0f, 0.0f, 0.0f, 1.0f);
```

Ce mouvement de 5 degrés d'amplitude donne vie au personnage.

5.2 Contrôles manuels

L'utilisateur peut interagir avec les animations via :

- **ESPACE** : Active/désactive l'animation de Pikachu
- **R** : Active/désactive la rotation automatique de la scène
- Les modifications sont confirmées par messages dans la console

Chapitre 6

Navigation et interaction

6.1 Système de caméra orbite

La caméra implémente un système orbite autour du personnage avec les contrôles suivants :

6.1.1 Zoom (touches Z/z)

- **Z majuscule** : Zoom avant (distance réduite de 0.5)
- **z minuscule** : Zoom arrière (distance augmentée de 0.5)
- Plage limitée entre 2.0 et 20.0 unités

6.1.2 Rotation (flèches et souris)

- **Flèches** : Rotation par incréments de 5 degrés
- **Souris** : Rotation libre avec clic gauche
- Limitation verticale : ± 89 degrés pour éviter les inversions

6.2 Gestion des événements

La gestion des entrées utilise le système de callbacks GLUT :

- `glutKeyboardFunc` : touches alphanumériques
- `glutSpecialFunc` : touches spéciales (flèches)
- `glutMouseFunc` : clics souris
- `glutMotionFunc` : déplacement souris
- `glutIdleFunc` : animation temps réel

Chapitre 7

Architecture du code

7.1 Organisation des fichiers

Le projet est structuré en trois fichiers principaux :

- `pikachu.h` : Déclaration de la classe `Pikachu`
- `pikachu.cpp` : Implémentation des méthodes de `Pikachu`
- `main.cpp` : Point d'entrée et gestion de l'application

7.2 Structure de la classe `Pikachu`

La classe `Pikachu` encapsule toute la logique de modélisation :

```
1 class Pikachu {  
2 private:  
3     // Textures et etats  
4     GLuint textureID, textureID2, backgroundTextureID;  
5     float animationAngle;  
6     int dimensionsTextures[2][2];  
7  
8     // Methodes de dessin des parties du corps  
9     void dessinerCorps();  
10    void dessinerTete();  
11    void dessinerOreilles();  
12    // ...  
13 };
```

7.3 Fonctions principales

- `chargerTextures()` : Initialisation des ressources
- `dessiner()` : Rendu complet du personnage
- `mettreAJourAnimation()` : Mise à jour des états d'animation
- `gererClavier()` : Traitement des interactions utilisateur

Chapitre 8

Résultats et difficultés

8.1 Résultats visuels obtenus

- Le rendu final présente un Pikachu reconnaissable avec :
- Formes proportionnées et respectueuses du design original
 - Textures appliquées correctement sur toutes les primitives
 - Éclairage mettant en volume le personnage
 - Animations fluides et naturelles
 - Navigation caméra intuitive et réactive

8.2 Difficultés rencontrées

8.2.1 Coordonnées de texture sur les sphères

La génération des coordonnées UV pour les sphères a nécessité des ajustements pour éviter les distorsions, particulièrement aux pôles.

8.2.2 Gestion hiérarchique des transformations

L'empilement des transformations (corps $\rightarrow$ tête $\rightarrow$ oreilles) a demandé une attention particulière dans l'ordre des opérations et la gestion des matrices.

8.2.3 Synchronisation des animations

La coordination entre la rotation automatique de la scène et l'animation de Pikachu a nécessité une gestion fine des états.

8.3 Solutions apportées

- Implémentation progressive avec tests incrémentaux
- Utilisation systématique de `glPushMatrix()/glPopMatrix()`
- Validation visuelle à chaque étape de développement
- Gestion robuste des erreurs de chargement

Chapitre 9

Conclusion

9.1 Bilan du projet

Le projet a permis de maîtriser les concepts fondamentaux de la synthèse d'image 3D :

- Modélisation géométrique par primitives
- Application et gestion de textures
- Éclairage et matériaux
- Animation et interaction temps réel
- Architecture logicielle pour applications graphiques

9.2 Améliorations possibles

- Implémentation de shaders GLSL pour des effets avancés
- Ajout de plus d'animations (clignement des yeux, mouvement des pattes)
- Chargement de modèles 3D externes pour comparaison
- Interface utilisateur graphique pour les paramètres
- Système d'ombres portées

9.3 Retour d'expérience

Ce projet a été l'occasion d'appliquer concrètement les notions vues en cours et de développer une compréhension approfondie du pipeline graphique. La modularité de l'architecture permet des extensions futures et le code source reste maintenable.

Chapitre 10

Annexes

10.1 Extraits de code significatifs

10.1.1 Génération de sphère paramétrique

```

1 void Pikachu::dessinerSphere(float radius, int segments) {
2     for (int i = 0; i < segments; ++i) {
3         float lat0 = M_PI * (-0.5f + (float)i / segments);
4         float lat1 = M_PI * (-0.5f + (float)(i + 1) / segments);
5         float z0 = sinf(lat0), zr0 = cosf(lat0);
6         float z1 = sinf(lat1), zr1 = cosf(lat1);
7
8         glBegin(GL_QUAD_STRIP);
9         for (int j = 0; j <= segments; ++j) {
10            float lng = 2.0f * M_PI * (float)j / segments;
11            float x = cosf(lng), y = sinf(lng);
12            // Calcul des coordonnees de texture et des sommets
13        }
14        glEnd();
15    }
16 }

```

10.1.2 Gestion des textures JPEG

```

1 bool Pikachu::chargerImageJpeg(const char* filename, GLuint &textureID,
2                               int &largeur, int &hauteur) {
3     // Ouverture fichier et decompression JPEG
4     struct jpeg_decompress_struct cinfo;
5     // ... traitement libjpeg
6     // Creation texture OpenGL
7     glGenTextures(1, &textureID);
8     glBindTexture(GL_TEXTURE_2D, textureID);
9     glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, largeur, hauteur,
10                0, format, GL_UNSIGNED_BYTE, data);
11 }

```

10.2 Références bibliographiques

- Documentation OpenGL : <https://www.opengl.org/documentation/>
- Documentation GLUT : <https://www.opengl.org/resources/libraries/glut/>
- Librairie libjpeg : <http://www.ijg.org/>